

Project Report: Lahore Fresh Bakes

Student: Amna Zulifqar

Project: Responsive business website with admin panel

Technologies: PHP, MySQL, HTML, CSS

Business: Lahore Fresh Bakes, a local bakery in Johar Town, Lahore

1. Business Problem

Lahore Fresh Bakes is a small local bakery in Johar Town, Lahore. It sells custom cakes, fresh breads, cupcakes, and dessert boxes for birthdays, weddings, and small events. The bakery is run by the owner with a small team, and at present the business has no online presence of any kind.

Because of this, every single customer question has to be answered in person or on the phone. A customer who wants to order a birthday cake has to either visit the shop or call the owner to ask three basic things: what items are available, what the price is, and whether the order can be ready in time. These are the same questions asked again and again, many times every day.

This creates problems for both sides.

For the customer, there is no easy way to see what the bakery offers. Prices are not written anywhere the customer can check, so the customer has to call just to find out if an item fits their budget. If the shop is busy or closed, the customer gets no answer at all and often orders from another bakery instead. The customer also cannot compare items calmly at home before deciding.

For the bakery owner, the repeated phone calls take time away from actual baking. During busy periods such as Eid and the wedding season, the owner is answering the same price questions while cakes are waiting to be finished. Orders taken over the phone are written on paper or remembered from memory, so details like pickup time or cake message are sometimes lost or mixed up between two customers. There is also no record of enquiries, so if a customer calls and does not order immediately, that enquiry disappears completely and cannot be followed up later.

Another problem is that the bakery has no way of showing who they are. A bakery is a trust business: customers are ordering food for family events, so they want to know that real, experienced people are preparing it. At the moment the owner cannot show the team or their experience to anyone who has not already visited the shop in person.

In short, the business needs a way to present its products and prices publicly, to collect customer enquiries in a form that does not get lost, and to do all of this without the owner needing to edit code every time a price or a product changes.

2. Proposed Solution

To solve this problem I developed a responsive website with a secure admin panel using PHP and MySQL. The solution is divided into two parts: a public website for customers, and a private admin panel for the owner.

2.1 Public website

The public side has four pages:

- **Home** — an introduction banner explaining what the bakery does, with the three newest products shown automatically.
- **About** — the bakery's background, its business problem and solution, and a "Meet Our Team" section showing the staff.
- **Products** — a full list of every product with its category, description, price in PKR, and photo.
- **Contact** — a contact form where a customer submits their name, email, phone number, and order details.

Prices and products are read from the database, not typed into the HTML. This means the owner never has to touch any code: whatever is entered in the admin panel appears on the public pages immediately.

The contact form solves the lost-enquiry problem. Every submission is saved permanently into a database table, so the owner can open the admin panel later and see the full list of enquiries with the date of each one. No enquiry is lost because the shop was busy or closed.

2.2 Secure admin panel

The admin panel is protected by a login system. Passwords are never stored as plain text in the database. Instead, only a hash of the password is stored, and PHP's `password_verify()` function checks the typed password against that hash. Once the login succeeds, the admin's id is stored in the PHP session, and a helper function called `require_admin()` runs at the top of every admin page. If a visitor who is not logged in tries to open any admin page directly by typing its address, they are redirected to the login page before any private data is printed on the screen.

Inside the panel the owner can:

- See a dashboard with the total number of products, categories, messages, and team members.
- **Manage products** with full CRUD: add a new product, edit an existing one, delete one, and view the complete list.
- **Manage team members** with full CRUD: add a member with their name, position, short biography, and photo, then edit or delete them later.
- Read all customer messages received through the contact form.

Team member management is what makes the About page useful. The owner adds a staff member once in the admin panel, and that person appears immediately in the "Meet Our Team" section with their photo, role, and biography. This lets the bakery show the real people behind the products, which builds the trust that a food business depends on. If a member has no photo uploaded yet, the website shows the first letter of their name in a circle instead, so the layout never looks broken.

2.3 Image upload

Both products and team members support image upload, handled by one shared function so the same validation rules apply everywhere. Three checks are performed before any file is saved:

1. **File type** is read from the actual contents of the file using `mime_content_type()`, not from the file extension. This is important because a file extension can be renamed. Only JPG, PNG, and WEBP are accepted.

2. **File size** must be under 2MB, so the server is not filled with very large images.
3. **File name** is regenerated using `uniqid()` , so two customers uploading files with the same name cannot overwrite each other's images.

If a file fails these checks, a clear message such as "Only JPG, PNG, and WEBP images are allowed." is shown and no incomplete record is saved to the database. When a record is deleted, its image file is also removed from the uploads folder so unused files do not build up over time.

2.4 Database design

The database contains five tables:

Table	Purpose
<code>admins</code>	Login accounts for the admin panel
<code>categories</code>	Product groups: Cakes, Breads, Desserts
<code>products</code>	Bakery items, each belonging to one category
<code>messages</code>	Enquiries submitted through the contact form
<code>team_members</code>	Bakery staff shown on the About page

The `products` table is related to the `categories` table through a foreign key on `category_id` , set to `ON DELETE CASCADE` . This means every product must belong to a real, existing category, and if a category is ever removed its products are removed with it, so the database can never be left with products pointing at a category that no longer exists.

All database queries in the project use PDO prepared statements with `?` placeholders instead of putting user input directly into the SQL text. This protects the site from SQL injection. Every value printed back into a page passes through an `e()` helper function which escapes HTML, so a customer cannot inject scripts through the contact form.

2.5 Responsive design

The layout uses CSS Grid and a media query at 820px. On desktop, product and team cards are shown three to a row and the admin panel has a fixed sidebar on the left. On a mobile screen, the cards stack into a single column, the navigation menu wraps, and the admin sidebar moves above the content. The same stylesheet serves both the public pages and the admin panel, which keeps the design consistent.

3. AI Tools Used

I used AI tools at several stages of this project, mainly as a helper rather than as a replacement for writing the project myself.

Layout and design suggestions. I used AI to get ideas for the page structure, such as how to arrange the hero section on the home page and how to present the team members as round-photo cards. I described what I wanted and used the

suggestions as a starting point, then adjusted the colours to a warm brown and cream theme that suits a bakery.

Finding and fixing code errors. This was the most useful part. When a page showed a blank screen or a PHP error I could not understand, I used AI to explain what the error message meant. For example, when my edit form was clearing the product image every time I saved without choosing a new file, AI helped me understand that I needed to pass the existing file name through a hidden field so the upload function could keep it.

Content writing. I used AI to help improve the wording of the descriptive text on the About page and the product descriptions, so the English reads more professionally than my first draft.

Understanding security concepts. I used AI to understand *why* certain things are done rather than just copying them: why passwords are hashed instead of stored directly, why prepared statements prevent SQL injection, and why checking the MIME type is safer than checking the file extension. This helped me explain my own code during the demonstration.

After using AI for these parts, I arranged and tested the whole project myself according to the final project requirements, and I made sure I could explain how every file works.

4. Challenges Faced

Moving from static pages to a database. My first version was plain HTML pages with the products typed directly into the code. The main challenge was changing this so the products came from MySQL instead. I had to learn how to connect PHP to MySQL with PDO, how to write a `SELECT` query, and how to loop through the results with `foreach` to print one card per row. Understanding that the page is generated fresh from the database on every visit, rather than being a fixed file, was the biggest change in how I was thinking about the project.

Getting the JOIN query right. Each product stores only a `category_id` number, so my first products page showed "1" and "2" instead of "Cakes" and "Breads". I had to learn how to use a `JOIN` to read the category name from the other table in the same query, and how to use `AS category_name` to give the joined column a clear name.

Image upload validation. Image upload took the longest to get right. My first version accepted any file, which meant someone could upload a `.php` file and it would be saved into the uploads folder. I learned that checking the file extension is not enough because a file can simply be renamed, so I changed the code to read the real file type from the file contents. I also hit the problem that two uploads with the same file name would overwrite each other, which I fixed by generating a unique name for every upload.

Keeping the existing image when editing. When I first built the edit forms, saving a product without choosing a new image would wipe the existing photo, because the file input was empty. I solved this by adding a hidden `old_image` field to the form and passing it into my upload function, which returns the old file name when no new file was chosen. The same fix works for both products and team members because they share one upload function.

Protecting the admin panel properly. At first I only put a login check on the dashboard, and I later realised I could still open the products page directly by typing its address, which showed private data. I fixed this by putting `require_admin()` inside the shared admin header file, which every admin page includes, so no admin page can be reached without logging in. I tested this by opening each admin page in a browser while logged out and confirming that all of them redirect to the login page.

Deleting a record without leaving files behind. When I deleted a product, its image stayed in the uploads folder forever as an unused file. To fix this I had to read the image name from the database *before* running the `DELETE` query,

because once the row is gone the file name is gone with it. Getting this order correct was a small but important detail.

Responsive layout. Making the same design work on both a phone and a laptop needed several attempts. Three-column card grids that looked good on desktop were far too narrow on a phone. I used CSS Grid with a media query at 820px so the columns collapse to a single column, and I had to remember to include the viewport meta tag, without which the mobile browser ignores the media query completely.

5. Conclusion

This project gives Lahore Fresh Bakes a complete, working online presence that directly addresses the problems the business started with. Customers can now see every product with its price and photo at any time of day without phoning the shop, and they can send an order enquiry that is saved permanently instead of being lost. The owner can update products, prices, and staff details independently through the admin panel without needing a developer or touching any code.

The finished project meets all of the required criteria: a four-page responsive website for a real local business, a secure admin panel with session-based login and hashed passwords, full CRUD functionality for both products and team members, validated image upload, and a MySQL database of five tables joined by a foreign key relationship.

If I continued developing this project, the next features I would add are an online ordering system with order status tracking, and a gallery of past custom cakes so customers can see previous work before ordering.